CURS DE POSTGRESQL

PostgreSQL 17



PSIG
Implementació, gestió i formació SIG

Carlos López Quintanilla
Consultor SIG

maig de 2026
Barcelona

Calendari i horari

Lloc:

Edifici MediaTIC- Cibernarium

Els dies (5):

- **Dijous 7 de maig**
- **Dilluns 11 de maig**
- **Dimarts 12 de maig**
- **Dimecres 13 de maig**
- **Dijous 14 de maig**

Horari:

- **De 15:3 a 19:30 horas (4 hores)**

Total 20 hores de curs



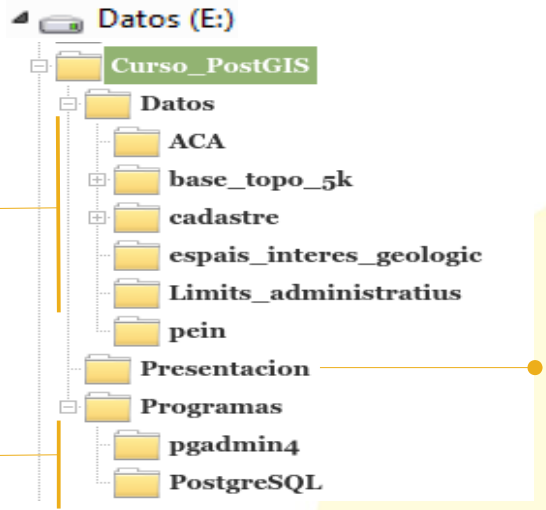


Hay que hacer un descanso

Con descanso se rinde más!

Material del curs

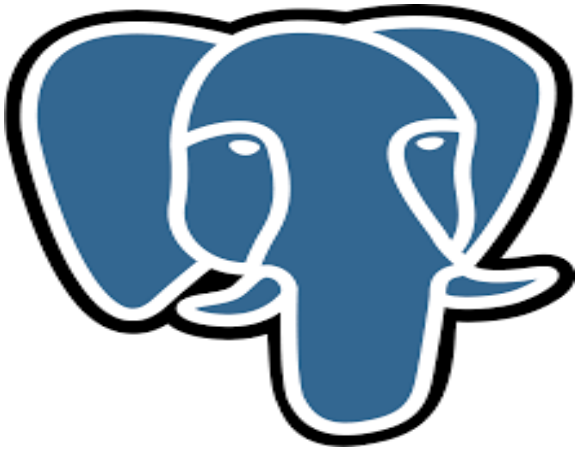
Dades per realitzar els exercicis (shp, tif i altres)



Aquesta presentació

Programes i aplicacions

Ens presentems !!!



- Nom
- Formació
- Professió
- Perqué PostgreSQL
- Experiències amb altres bases de dades

Índex

1. Introducció a PostgreSQL
2. Estructura i Arquitectura de PostgreSQL
3. Comandes bàsiques d'SQL en PostgreSQL
4. Funcions i procediments
5. Indexació i optimització
6. Administració de la Base de Dades
7. Funcions avançades

Aquesta obra està subjecta sota Licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional

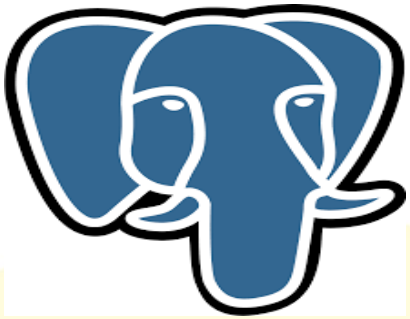
Índex

1. Introducció a PostgreSQL

1.1 Història i característiques de PostgreSQL
1.2 Instal·lació i configuració inicial
1.3 Eines d'administració (pgAdmin, psql)

1.1. Història i característiques de PostgreSQL

PostgreSQL



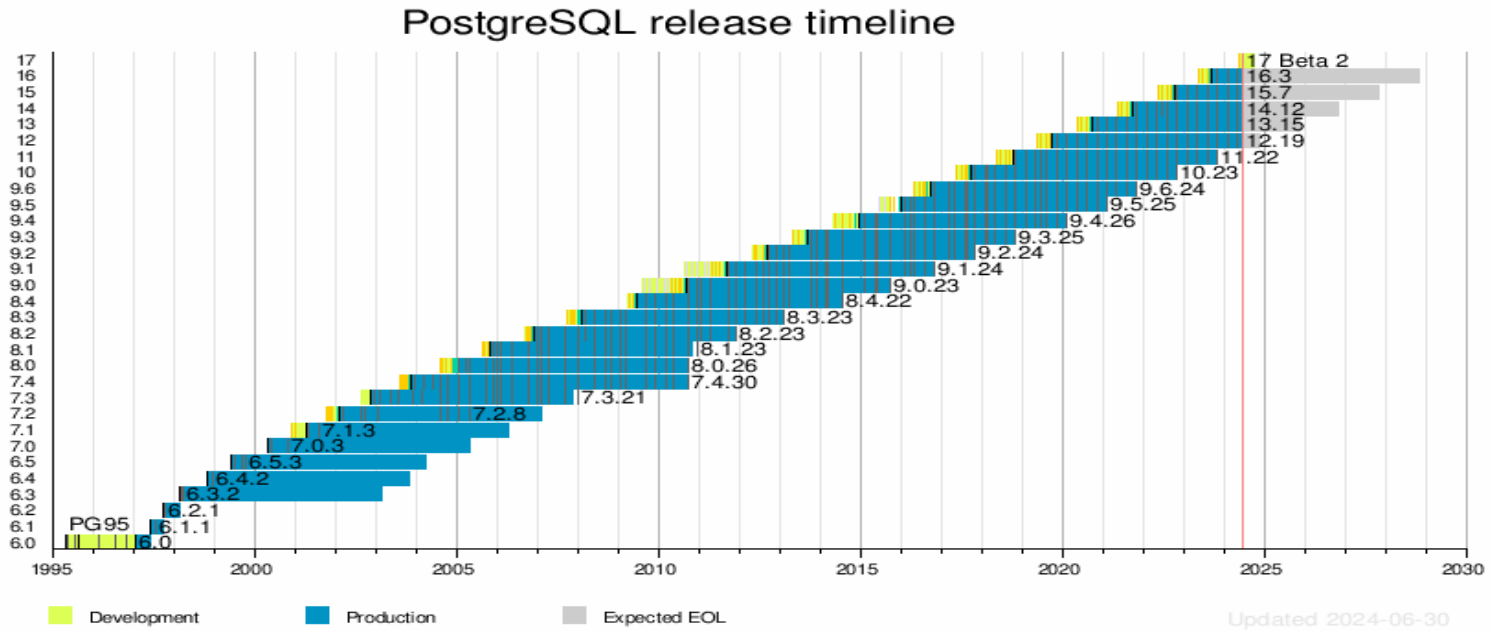
- PostgreSQL es programari lliure que implementa un sistema de gestió de bases de dades relacional.
- Escrit en C i multiplataforma

- Característiques principals:
  - Alta concurrencia
  - Amplia varietat de tipus de dades (text il·limitat, geometries, tipus propis...).
  - Vistes
  - Integritat transsacional
  - Disparadors (triggers en anglès)
  - Funcions en múltiples llenguatges

  Més informació a la Viquipèdia:
  https://ca.wikipedia.org/wiki/PostgreSQL

PostgreSQL

Historial de versions



1.2 Instal·lació i configuració inicial

Instal·lació

- SO Windows:
  Tutorial per fer la instal·lació:
  https://www.youtube.com/watch?v=_EgtELrjLO4

- SO Linux
  Manual d'instal·lació en Linux:
  https://github.com/carlospsig/protocol_install_qgis_ubuntu/blob/master/install_postgis.sh

  Apunteu els paràmetres de connexió:

host=localhost
port=5432
dbname=postgres
user=postgres
password=??

Configuració

- SO Windows:
  C:\Program Files\PostgreSQL\15\data\pg_hba.conf
  C:\Program Files\PostgreSQL\15\data\postgresql.conf

- SO Linux
  /etc/postgresql/15/main/postgresql.conf
  /etc/postgresql/15/main/pg_hba.conf

Configuració

- **Postgresql.conf**:

uncomment the next line
listen_addresses = ''*
exit and save

- **pg_hba.conf**

Add new line at the last line, to open all IP to acces to data base
host all all 0.0.0.0/0 scram-sha-256
exit and save

You have to restart the postgresql service to see the configuration changes
service postgresql restart

pg_service.conf

Para conectar con una base de datos podemos definir los parámetros de conexión con un fichero local de tipo INI.

Primero hay que declarar una variable de entorno (usuario o sistema):

- Nombre de la variable = PGSYSCONFDIR
- Valor de la variable = ubicación del fichero pg_service.conf

Atención:
En Windows, hay que guardar pg_service.conf en formato Unix.
Para hacerlo con el Notepad++ ir a:
Editar --> Conversión fin de línia --> Convertir en formato UNIX. Guardar

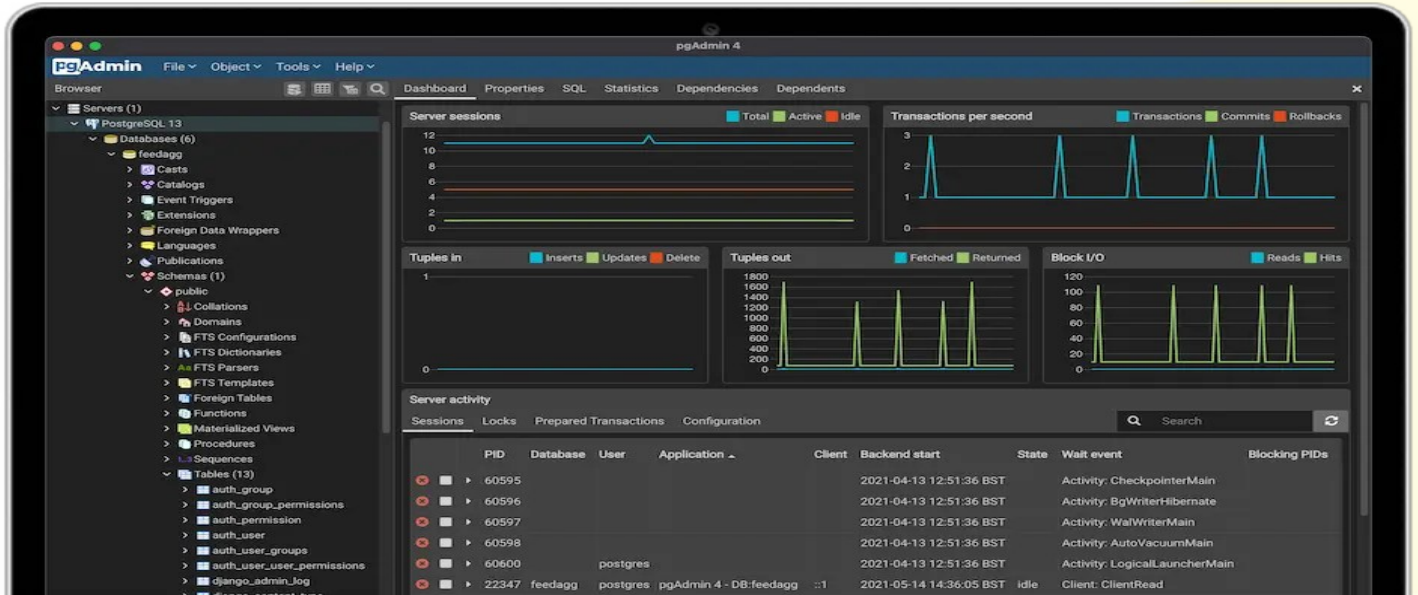
Contenido del fichero pg_service.conf

```
[gis_curso]
host=localhost
port=5432
dbname=gis_curso
user=postgres
password=postgres
```

1.3. Eines d'administració (pgAdmin, psql)

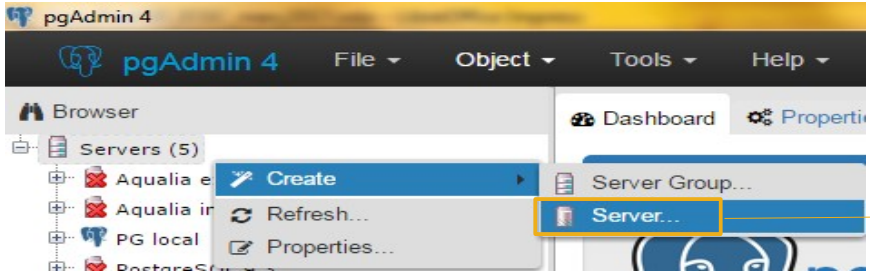
Administració – pgAdmin 4

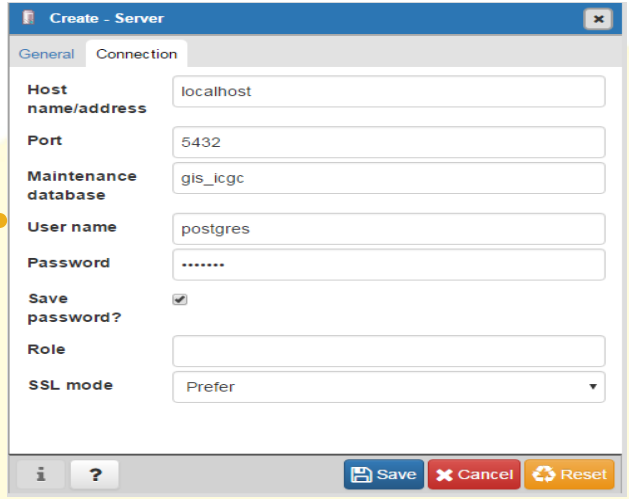
- **pgAdmin 4** es l'aplicació més popular programari lliure per administrar i desenvolupar bases de dades PostgreSQL.



pgAdmin 4 connexió a PostgreSQL

Per connectar amb una base de dades de tipus PostgreSQL
Farem servir això: Servers >> Create >> Server...





Name: el que queráis
Host: localhost
Port: 5432
DataBse: postgres
Username: postgres
Password: ?????

Webs de referencia

Página web oficial de PostgreSQL:
https://www.postgresql.org/
Descarga del programa, documentación, manual, ejemplos de uso



Página web oficial de pgAdmin:
http://pgadmin.org/
Descarga del programa, documentación, código, etc...



Tutorial SQL w3school (solo querys):
https://www.w3schools.com/sql/



Índex

2 Estructura i arquitectura de PostgreSQL

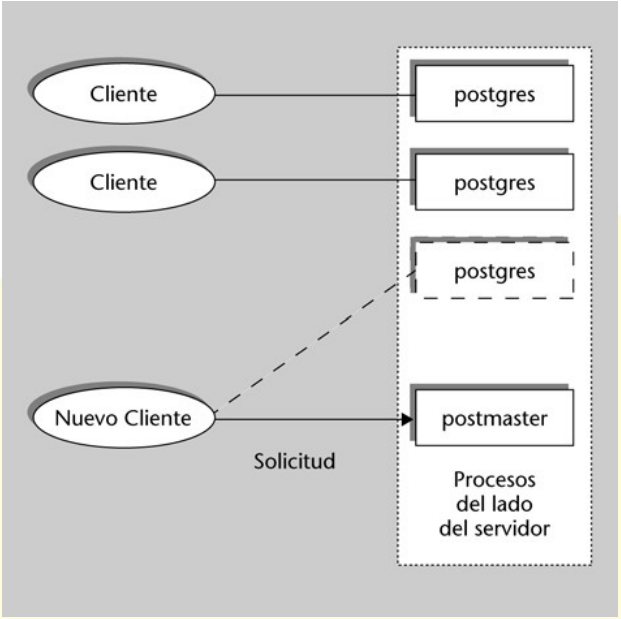
2.1 Arquitectura de bases de dades
2.2 Tipus de dades en PostgreSQL
2.3 Creació i gestió de bases de dades i taules

2.5 Arquitectura de base de dades

Arquitectura

PostgreSQL es una base de dades amb estructura **client-servidor**

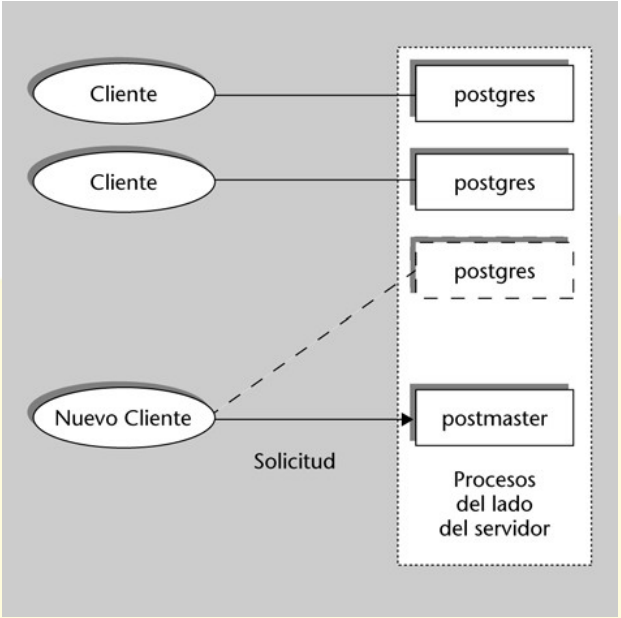
Una instancia PostgreSQL se conoce como un cluster de base de datos, que es una colección de bases de datos



Arquitectura

Servidor seria el mateix PostgreSQL

Client:
- pgAdmin
- DBeaver
- psql
- QGIS



2.2 Tipus de dades en PostgreSQL

Tipus de dades natives SQL

Tipos de datos del estándar SQL3 en PostgreSQL		
Tipo en Postgres	Correspondiente en SQL3	Descripción
bool	boolean	valor lógico o booleano (true/false)
char(n)	character(n)	cadena de carácteres de tamaño fijo
date	date	fecha (sin hora)
float4/8	float(86#86)	número de punto flotante con precisión 86#86
float8	real, double precision	número de punto flotante de doble precisión
int2	smallint	entero de dos bytes con signo
int4	int, integer	entero de cuatro bytes con signo
int4	decimal(87#87)	número exacto con 88#88
int4	numeric(87#87)	número exacto con 89#89
money	decimal(9,2)	cantidad monetaria
time	time	hora en horas, minutos, segundos y centésimas
timespan	interval	intervalo de tiempo
timestamp	timestamp with time zone	fecha y hora con zonificación
varchar(n)	character varying(n)	cadena de carácteres de tamaño variable

Tipus de dades PostgreSQL

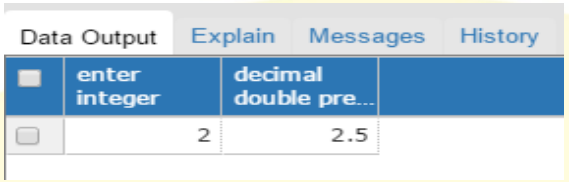
Tipos de datos extendidos en PostgreSQL	
Tipo	Descripción
box	caja rectangular en el plano
cidr	dirección de red o de *host* en IP versión 4
circle	círculo en el plano
inet	dirección de red o de *host* en IP versión 4
int8	entero de ocho bytes con signo
line	línea infinita en el plano
lseg	segmento de línea en el plano
path	trayectoria geométrica, abierta o cerrada, en el plano
point	punto geométrico en el plano
polygon	trayectoria geométrica cerrada en el plano
serial	identificador numerico único

Conversió de tipus de dades

Por suerte, SQL permite cambiar el tipo de dato, cambiando su comportamiento. Cambiar el tipo de un dato se llama **casting**
Vigilar por que, si conversión puede fallar si esta no es posible.

Para convertir un tipo de datos, hay 2 opciones:
:: → dato::tipo – versión Postgres
Cast → cast(dato as tipo) – versión SQL general

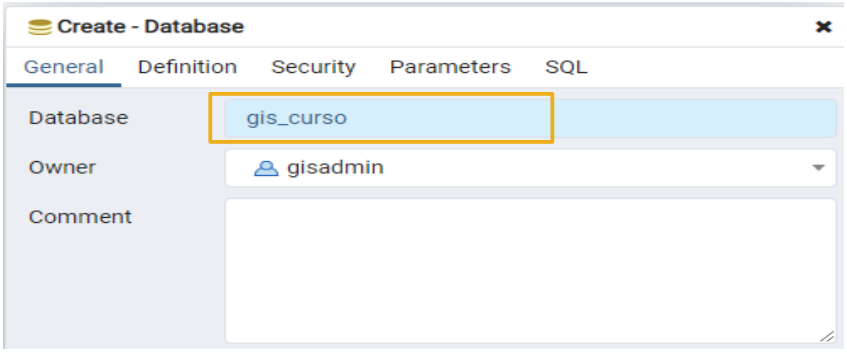
select 5/2 as enter, 5::float/2 as decimal;

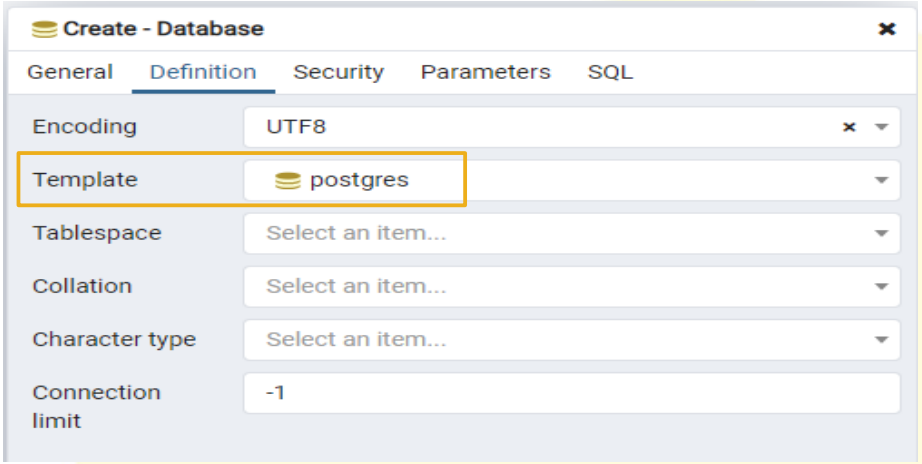


2.3 Creació i gestió de bases de dades i taules

Generar la base de dades







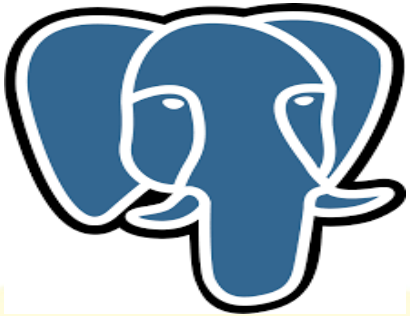
Índex

3 Comandes bàsiques d'SQL en PostgreSQL

- 3.1 Comandes SQL bàsiques
- 3.2 Funcions de PostgreSQL
- 3.3 Subqueries, unions, i joins
- 3.4 Gestió d'errors

3.1 Comandes SQL bàsiques

Introducció a SQL

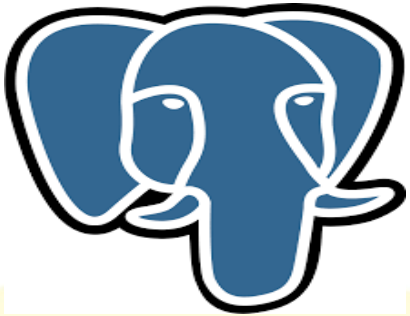


Structured Query Language (SQL) es un llenguatge de programació dissenyat per a treballar amb bases de dades.

La primera versió es va fer l'any 1986 i la darrera (SQL3) va sortir l'any 1999. La darrera ampliació es va fer l'any 2006.

Moltes bases de dades incorporen sentències pròpies al llenguatge base, la base funciona per a totes. Nosaltres farem servir algunes funcions pròpies de la versió de PostgreSQL.

Comentaris



Per comentar el codi tenim dues opcions

-- Dos guions fan que la linia sigui un comentari

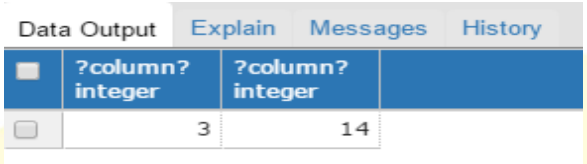
/
Una barra i un asterisc fan que sigui comentari fins a trobar encontrar un asterisc i una barra.
Això servirà per a comentar múltiples línies
**/*

Select

La instrucció ***Select*** selecciona dades que son enviats a l'usuari en forma de taula.
Es la instrucció més bàsica i la més complexa al mateix temps, ja que conté moltes variants i possibles modificacions.

Al final de cada sentència haurem d'afegir **punto i coma** (;), això serveis para donar-li la instrucció al PostgreSQL per a que s'executi:

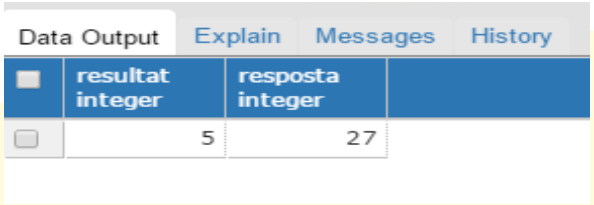
```
select 3, 12+2;
```



As

A l'exemple anterior podem veure que las columnes no tenen nom. Per canviar el nom de las columnes, farem servir ***AS***, això canviarà el nom d'aquestes.

Ho farem servir per crear noms o **àlies**




```
select 3+2 as resultado, 6+21 as respuesta;
```

From

Una base de dades sirve para guardar y leer datos.
Vamos a hacer ahora esto último.

FROM nos permite acceder a los datos de una tabla de la base datos. El contenido de las columnas resultantes será ejecutado para todas las filas de la tabla. Podemos especificar la columna de cada interación por su nom.

```
select nommuni, geom from limit_admin.munis_poly;
```

From

A veces nos interesa coger todos los campos y valores de una tabla. Esto se puede hacer utilitzando un asterisco.

```
select * from limit_admin.munis_poly;
```

También se pueden hacer operaciones con los datos de cada columna.

```
select id, id+5 from limit_admin.munis_poly;
```

Sensibilidad a mayúsculas

El lenguaje SQL solo distingue entre mayúsculas y minúsculas en dos casos:
1- Valores literales (como por ejemplo text)
2- Nombres identificados entre comillas dobles (“)

También ignora las nuevas líneas, espacios extras y tabulaciones.

Hasta ahora hemos mostrado las sentencias con el formato mayúsculas para las instrucciones y minúsculas para los nombres. Esto es debido a motivos históricos.

Lo más importante es utilizar siempre el mismo sistema.

```
SELECT id, id+5 FROM limit_admin.munis_poly;
```

Sensibilidad a mayúsculas

Recomendación!!

Definir todo lo que sea estructural con **minúsculas**,
- nombre de base de datos,
- esquemas,
- tablas,
- vistas,
- campos,
- roles,
- etc...

Where

La instrucción where introduce condicionales al select.
Solo se devuelven las filas que cumplan la condición escrita en el where. Esta puede ser cualquier expresión que sea de tipo booleano.

```
select columna
   from basic.taula
   where columna > 3
;
```

Operaciones booleanas

Op	Definición
a = b	Igual que
a <> b a != b	Diferente que
a > b	Más grande que
a >= b	Más grande o igual que
a < b	Más pequeño que
a <= b	Más pequeño o igual que
not a	$\bar{a}$ (álgebra de Boole)
a or b	a·b (álgebra de Boole)
a and b	a+b (álgebra de Boole)

Where

Pequeño ejercicio:

- ¿Qué id tiene la comarca del Bages?
- ¿Cuántos municipios hay en la comarca del Baix Llobregat?
- ¿Cuántos municipios tienen más de 1000 metros de altitud y más de 1000 habitantes?
- ¿Cuántos municipios tiene más de 200 km2?

Order by

Order by se pone después del *where* y ordena las filas según los campos y criterios elegidos.

Ejemplos:

```
select id, ST_Area(geom) as area
   from limit_admin.comarques
   order by area, id;
```

```
select id, ST_Area(geom) as area
   from limit_admin.comarques
   order by area desc, id;
```

En el primer caso, se ordena de forma ascendente (de menor a mayor) según el área de la comarca (en m^2). En caso de empate se desempatada por id ascendente.

Notas:
Por defecto order by es ascendente. Aunque se puede utilizar ASC para ser explícito.

Carácter y textos serán ordenados alfabéticamente.

Limit

Limit hace que el resultado final devuelva las primeras filas del resultado que tendría si no le ponemos límit.
Esto permite hacer una carga parcial de una tabla muy grande, y hacer testeo y pruebas con una muestra parcial.

```
select id, ST_Area(geom) as area
   from limit_admin.comarques
   order by area, id
   limit 10
;
```

Pequeño ejercicio:
- ¿Cuáles son las 5 comarcas de Catalunya con más área (en km²)?
- ¿Cuáles son los 10 municipios de Catalunya con más habitantes?

Group by

Agrupa las columnas en grupos, definidos por las columnas deseadas.

Todas las columnas de la tabla final han de estar definidas en los grupos, o ser una función agregativa o bien no ser utilizadas.

```
select codiprov, sum(habitants) as habitants
   from limit_admin.munis_poly
   group by codiprov
;
```

Pequeño ejercicio:

- ¿Cuántos municipios hay en cada comarca?
- ¿Cuántos habitantes hay en cada provincia de Catalunya?

Agregación

La agregación consiste en utilizar más de una fila para calcular un resultado final.

Los casos más habituales son.:
- Sumatorios
- Contar files
- Cálculos estadísticos (como por ejemplo la media, mín. , máx.)

Cuando se utiliza la agregación hay que estar alerta con los campos que se agregan (si se intenta utilizar una columna sin agregar, no sabrá que valor coger).

Cuando se hace una agregación, las instrucciones WHERE, ORDER BY, LIMIT, etc. afectan a los datos antes de agregarse.

```
select sum(value)
   from exemple
```

Having

Como ya hemos visto, *where* afecta a los valores antes de la agregación. Para hacerlo después se puede utilizar *having*.

```
select codiprov, ST_Union(geom) as geom
    from limit_admin.munis_poly
    group by codiprov
    having codiprov = '08'
;
```

Ejemplo de select simple (todo lo que hemos visto)

Revisar l'ordre de las funciones

```
select codiprov , sum(ST_Area(geom)) as area
   from limit_admin.munis_poly
   where ST_Area(geom) > 10000000
   group by codiprov
   having codiprov in ('08','17', '25')
   order by area desc
   limit 2
;
```

Així es com PostgreSQL ho procesa



3.2 Funcions de PostgreSQL

Funciones de PostgreSQL

PostgreSQL también tiene sus propias funciones. Aquí algunas de las más importantes:

- coalesce(a, b, ...): Devuelve el primer valor no null del listado o null si todo lo son.
- nullif(a, b): Si *a* igual a *b*, devuelve null. Si no, devuelve *a*.
- abs(a): Valor absoluto d'*a*.
- round(a): Redondea *a*.
- lower(a): Convierte *a* a minúsculas.
- date_trunc(medida, a): Trunca la fecha/hora *a* a la medida *medida*.

Funciones de PostgreSQL (última versión): www.postgresql.org/docs/current/static/functions.html

3.3 Subqueries, unions, i joins

Join (1)

Permite agrupar dos (o más) tablas en una sola en la misma query. La condición que agrupa las diferentes filas de las tables es la condición ***on***.

```
select m.nommuni, p.nomprov
   from limit_admin.munis_poly as m
      left join limit_admin.provincies as p
         on m.codiprov = p.codiprov
   group by m.nommuni, p.nomprov
;
```

Como se puede ver en el ejemplo se pueden utilizar alias por las tablas. Esto es muy útil para no tener que reescribir el nombre de la tabla entera cada vez.

```
    limit_admin.munis_poly as m
```

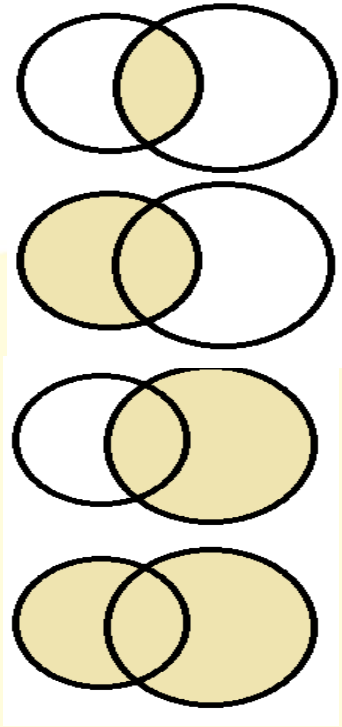
Tipus de Join (2)

Inner join
Las filas se juntan solo si la condición se cumple.

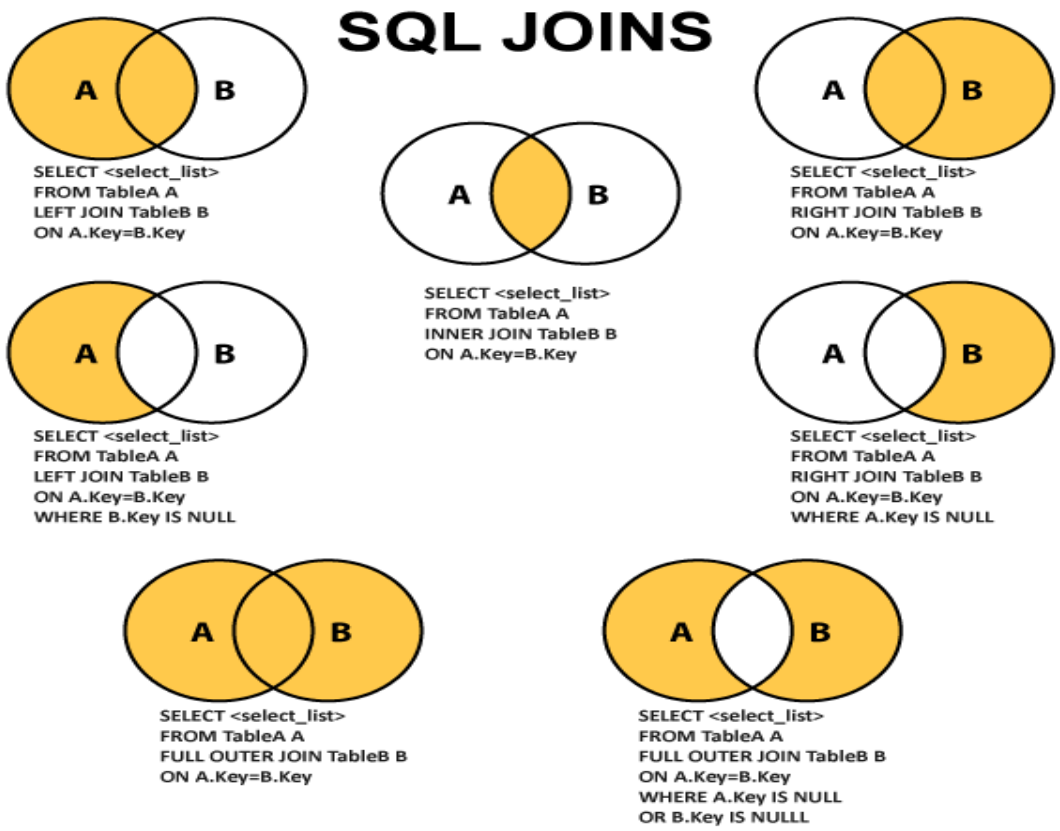
Left join
Las filas se juntan si la condición se cumple. Además se mantienen las filas de la izquierda (primera tabla) con los valores de la tabla de la derecha como null.

Right join
Equivalente al anterior pero girando el orden de las tablas.

Full join
Se muestran las filas que se pueden cruzar con la condición. Además és todas las otras (tanto las de la derecha como las de la izquierda) que no es pueden emparejar con ninguna fila, dejando los valores de la otra tabla como null (como si fuese *right join* y *left join* a la vez).



Join (3)


SQL JOINS
A
B
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key=B.Key
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key=B.Key
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key=B.Key
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key=B.Key
WHERE B.Key IS NULL
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key=B.Key
WHERE A.Key IS NULL
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key=B.Key
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key=B.Key
WHERE A.Key IS NULL
OR B.Key IS NULLL

Join (4)

Se pueden encadenar múltiples JOIN ... ON ... JOIN ... ON ... para juntar más de dos tables. La manera de como se juntaran dependerá de la relación especificada en ON.

Los joins son muy comunes en los modelos relacionales ya que los datos finales suelen estar en otra tabla (para evitar repeticiones).

Ejercicio:
¿Cuantos municipios están en el interior del Parc Natural del Montserrat?

Ejemplo:

```
select *
   from taula1 as p
      left join taula2 as s
         on p.t2 = s.id
      left join taula3 as t
         on s.t3 = t.id
   where p.id > 100
   limit 100
;
```

Subquery (1)

Las subquerys permiten utilizar el resultado de una query en otra query (de aquí su nombre).

Estas subquerys pueden servir de tablas hijas (si es limita a una fila por cada subquery) o valores (si tienen una hija y una columna).

```
-- Provincia que tiene el municipi más pequeño
select p.id, p.nomprov
   from limit_admin.provincies as p
   where codiprov = (
      select codiprov
         from limit_admin.munis_poly
         order by ST_Area(geom) asc
         limit 1
   );
```

Nota:
La subquery no acaban en punto y coma (;) ya que el comando no se ejecuta hasta que la query principal se acaba.

Union

La instrucción UNION une las filas de los resultados de múltiples queries en la primera. Para hacerlo es necesario que las columnas tengan el mismo tipo de datos. Mantendrán el nombre de las columnas de la primera query.

Para aplicar instrucciones al resultado se tiene que usar una subquery.

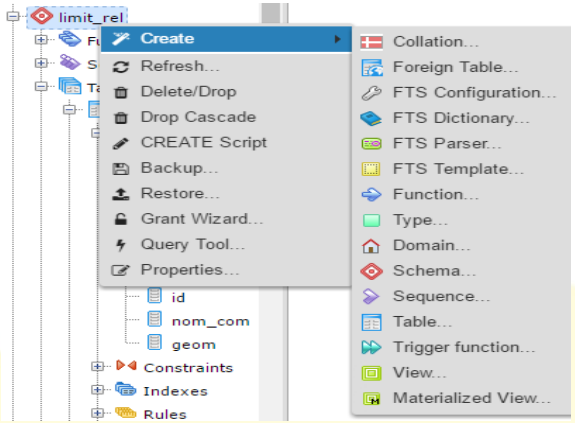
```
   select 1 as resultat, id from tabla 1
union
  select 2, id from tabla 2
;
```

Ejercicio:
Listar los nombres de municipios y de espais del PEIN en una sola tabla

Create

Crea un objecto. Cada objecto tiene su estructura particular de la instrucción.

Recomendable hacerlos desde el programa pgAdmin excepto si se quiere guardar la instrucción en un fichero o bien si haciéndolo con el pgAdmin es más lento.



Ejemple:

```
 -- create table
create schema test_schema;
create table test_schema.test (
                id serial primary key,
                geom geometry('MultiPolygon', 25831)
);
```

Alter table

Utilizaremos esta instrucción para modificar la estructura de una tabla existente.
Modificar una columna actual o añadir una nueva.

	id [PK] serial	n integer	num double precis
*			

Ejemplo:

```
 -- alter table
alter table test_schema.test add column num float;
```

Ejercicio
Añadir una nueva columna a la capa d'espais del PEIN para guardar la superficie en hectarees

Insert

Sirve para añedir filas a una tabla. No hace falta especificar todas las columnas, solo las que interesan (excepto que la tabla tenga alguna que no pueda ser null o sin valor por defecto).

VALUES sirve para llenar las tablas literalmente. También se puede llamar con un SELECT aunque es poco habitual.

INSERT permite usar un SELECT para conseguir los datos. Esto permite hacer copias y guardar cálculos en otra tabla.

Ejemplo:

```
 -- insert into
insert into test_schema.test(n) values   (5),    (15) ;
```

	id [PK] serial	n integer	num double precision
1	1	5	
2	2	15	
*			

Update

UPDATE actualiza valores de la tabla.

	id [PK] serial	n integer	num double precision
1	1	3	
2	2	15	
*			

Ejemplo:

```
 -- update
update test_schema.test
   set n = 3
   where id=1
;
```

Ejercicio:
Calcular la superficie de los espias del PEIN en hectarees

Delete

Elimina filas o registros de la tabla según el criterio elegido. Si no hay criterio, elimina todos los valores de la tabla (pero no la tabla en si).

Ejemplo:

```
 -- delete algunos registros
delete from test_schema.test
   where id < 2
;

-- delete todo el contenido de una tabla
delete from test_schema.test;
```

	id [PK] serial	n integer	num double precision
1	2	15	
*			

Drop & drop cascade

Drop elimina un objecto (tabla, función, vista, esquema, base de datos, etc.).

A veces nos podemos encontrar el caso que otro objeto necesita el que queremos eliminar. Podemos hacer dos coses:
1- Lo modificamos por que ya no necesitamos el objeto a eliminar, o
2- Lo eliminamos todo, el objeto y la dependencia.

Este segundo caso es muy habitual (por ejemplo eliminar un esquema con todo lo que tenemos dentro). Para hacerlo hay que llamar DROP CASCADE, que elimina todos los otros objetos que dependen del eliminado.
Como no requiere confirmación en la operación, hay que ir alerta cuando utilicemos DROP CASCADE ya que podría ser que elimines alguna cosa que no recuerdes que tuviese dependencia.

Ejemplo:

```
 -- drop
drop table test_schema.test;

 -- drop cascade
drop table test_schema.test
cascade;
```

Views

Son querys en forma de tabla. Externamente funcionan idénticamente a las tables. Esto permite hacer tables dinámicas que obtienen los valores de otras tables.

Muy útil para hacer una capa con información y cargarla en QGIS. La información puede estar guardada en diferentes lugares pero QGIS lo ve como una sola tabla con información.

Ejemplo:

```
create view test.vista_test as
   select * from test.test
;
```

Views

Recordar que QGIS necesita que las tables y las vistas de PostgreSQL tengan una clave primaria (PK)

Las tablas siempre tienen PK, pero las vistas no, antes de añadir la vista al QGIS hay que definir que campo o campos serán la PK

Ejercicio:
Convertir en vista los municipios que intersectan con Montserrat y cargar la vista en QGIS

Índex

4 Funcions i procediments

4.1 Funcions definides per l'usuari
4.2 Procediments emmagatzemats
4.3 Triggers i regles

4.1 Funcions definides per l'usuari

Funciones (1)

Durante el curso hemos trabajado con funciones de PostgreSQL.
Cada función hace un proceso.
Normalmente no sabemos como funciona internamente y no hace falta saberlo; solo nos interesa saber que hace.

Podemos definir nuestras propias funciones.
El formato básico es el siguiente:

```
create or replace nombre(param1 tipus1) returns tipus_r as $$
<codi>
$$ language llenguatge;
```

Funciones (1)

Por ejemplo:

```
create or replace suma(a int, b int) returns int as $$
   select a+b;
$$ language sql;
```

Los parámetros, en este caso *a* y *b*, se pueden llamar por su nombre o bien con *$n* donde *n* es la posición (seria *$1* y *$2*).

Funciones (2)

Si queremos devolver una tabla hay que definirla. Por ejemplo:

```
create or replace function prov(id bigint) returns table (
   id bigint,
   nom_prov text
) as $$
   select id, nom_prov
      from limit_rel.provincies
      where id = $1
   ;
$$ language sql;
```

Aquí utilizamos *$1* en lugar de *id* para evitar confusiones.
Las funciones con lenguaje SQL solo poden fer una sola query (con sus subquerys).
Para hacer múltiples querys se necesita un lenguaje procedural.

4.2 Procediments emmagatzemats

Programación procedural

Utilizando un lenguaje procedural, es decir, un lenguaje que pueda ejecutar múltiples comandos uno detrás de otro, se abren posibilidades que anteriormente no teníamos o eran muy complicado de hacer.

Nosotros nos centraremos en PL/pgSQL, ya que es el lenguaje procedural propio del PostgresSQL y se parece mucho al SQL.

Programación procedural

La estructura básica del *PL/pgSQL* es:

```
declare
   -- variables que se utilizarán
begin
   -- código
end;
```

En el caso de no utilizar ninguna variable, el apartado *declare* se omite.

Si nos fijamos en el ejemplo de la suma pero esta vez utilizando PL/pgSQL. En lugar de utilizar un *select* utilizaremos la instrucción *returns*, que devuelve el cálculo. Esto es debido a que no sabe que cálculo tiene que devolver, ya qu pueden haber diversos select.

```
create or replace suma(a int, b
int) returns int as $$
begin
   return a+b;
end;
$$ language plpgsql;
```

PL/pgSQL

PL/pgSQL añade muchas cosas al lenguaje SQL. Algunas de estas son:

- Variables
- Condicionales
- Bucles
- Tablas temporales
- Posibilidad de ejecutar strings como un comando

```
declare
  a int := 3;
  x int;
  r int := 0;
begin
  if a = 3 then
    perform a();
  elsif a = 4 then
    perform b();
  else
    perform c();
  end if;

for x in get_values() loop
  r := r+x;
end loop;

execute 'select a from ' ||
quote_indent('table'::text) || '
where x > 3;';
end;
```

Otros lenguajes

PostgreSQL permite el uso de varios lenguajes de programación: Python, TLC, Pearl y un lenguaje propio (PL/pgSQL). También permite ejecutar funciones de librerías dinámicas (.dll / .so). Además hay un repositorio no oficial para poder utilizar otros lenguajes como el Shell (bash), Java, PHP, etc.

Estos lenguajes se tienen que instalar, pero se utilizan de manera muy similar a PL/pgSQL:

```
create or replace suma(a int, b int) returns int as $$
# codi en una (petita) variant de Python 3
# alerta amb les tabulacions
return a+b;
$$ language plpython3u;
```

Do

SQL y PL/pgSQL permiten ejecutar código de otros lenguajes con la instrucción do. Esta es muy similar a la definición de las funciones:

```
do $codi1$
begin
-- código en plpgsql
do $codi2$
#código en python
$func2$ language plpython3u;
end;
$codi2$ language plpgsql;
```

Esta instrucción es muy útil para probar código de otros lenguajes como una query directamente.

4.3 Triggers i regles

Triggers (1) = Disparadores

Cada trigger ejecuta una función que es un poco especial cada vez que una condición se cumpla.
Muy útil para automatizar cambios, puede hacer desde calcular alguna columna hasta modificar otras tablas para hacer que los valores sean consistentes.

(before)
(after)

(instead of)

El trigger se puede activar de tres maneras (no todos los tipos los tienen):
- Antes de la instrucción, esto permite modificar los valores que esta aplica.
- Después de la instrucción, esto permite utilizar funciones que lean los cambios hechos.
- En lugar de la instrucción, esta no hace ningún cambio pero se ejecuta nuestra función.

(for each row)
(for each statement)

El trigger se pueden ejecutar
- Por cada fila.
- Una vez por sentencia.

Triggers (2)

Un ejemplo muy utilizado: calcular automáticamente el área de una geometría

```
-- create function
create or replace function test_schema.area_trigger() returns
trigger as $$
begin
    new.area := ST_Area(new.geom);
    return new;
end;
$$ language plpgsql;
```

```
-- create trigger
create trigger area_trigger
before insert or update of geom
on test_schema.test
for each row
execute procedure
test_schema.area_trigger();
```

Triggers (3)

Como hemos visto, la función devuelve el tipo *trigger* y, dentro la función, devuelve *new*.

new es una fila (tipo *record*) que se corresponde con la fila que está cambiando y como quedará si la modificamos con un trigger **antes** de que suceda, estamos cambiando lo que cambia.

Las funciones de trigger también tienen el tipo *old* que corresponde con la fila con los valores antes de cambiarlos.

new y *old* pueden no estar disponibles dependiendo de lo que haga el trigger. (si haces un *delete* no habrá ningún *new)*.

En el caso de utilizar un *for each statement*, no hay ni *new* ni *old*.

Triggers (4)

Una manera especial de utilizar los triggers es utilizarlos para editar el contenido de una vista. Se comporta, pues, como si la vista fuese editable.

Ejemplos:

```
-- Dada la vista
create view exemple.vista as
        select true as taula, id, geom from taula1
    union
        select false as taula, id, geom from taula2
;
```

Continua....

...

```
-- Preparar la vista para un insert
-- función
create function vista_insert() returns trigger as $$
begin
    if new.taula then
        insert into taula1(id, geom) select new.id, new.geom;
    else
        insert into taula2(id, geom) select new.id, new.geom;
    end if;
return new;
end;
$$ language plpgsql;
```

```
-- create trigger
create trigger vista_insert
    instead of insert on
exemple.vista
    for each row execute procedure
vista_insert()
;
```

Continua....

```
-- Preparar la vista per una actualització de geometria
-- Funció
create function vista_update_geom() returns trigger as $$
begin
    if new.taula then
        update taula1
        set geom=new.geom
        where id=new.id
        ;
    else
        update taula2
         set geom=new.geom
        where id=new.id
        ;
    end if;
return new;
end;
$$ language plpgsql;
```

```
-- Fem un trigger
create trigger vista_update_geom
    instead of update of geom on
exemple.vista
    for each row execute procedure
vista_update_geom()
;
```

Triggers (5)

Como punto final, los triggers activan otros triggers.
Puede ser muy útil pero alerta con los bucles infinitos (el PostgreSQL no lo revisa).

(Ejercicios)

- Crear un Trigger para actualizar la fecha de la última actualización
- Crear un Trigger para añadir el usuario que ha hecho la última actualización

Índex

5 Indexació i optimització

5.1 Tipus d'índexs i la seva creació

5.2 Optimització de consultes, ANALYZE

5.1 Tipus d'índexs i la seva creació

Create index

PostgreSQL provides the index methods B-tree, hash, GiST, SP-GiST, GIN, and BRIN. Users can also define their own index methods, but that is fairly complicated.

https://www.postgresql.org/docs/current/sql-createindex.html

Integridad de los datos

Dentro de una base de datos nos interesa asegurarnos de que todos los datos que haya sean posibles y tengan sentido. Es mejor que se queje antes de guardarlo que alguna cosa funcione mal posteriormente, o que no hayan errores más adelante. Por esto, se pueden generara restricciones que no permitan añadir o actualizar los datos si estos no cumplen las restricciones.

Ejemplo de restricciones:

```
create table exemple (
   id serial primary key,
   m int not null references model(id),
   valor float check (valor > 0)
);
```

Integridad de los datos

Hay diferentes tipos de restricciones en una o más columnas:
- Not null: El valor de la columna no puede ser null.
- Unique: El valor no puede ser repetido en una otra fila.
- Clave primaria (primary key): Identificador de la fila, implica la restricción not null y unique.
- Clave foránea (foreign key): La(as) columna(as) hace referencia a una clave primaria de otr tabla. Esta ha de existir.
- Check: una condición (booleana) a cumplir. (ej: valor > 100).

Estas restricciones no son excluyentes y se pueden añadir juntas.

Además, se puede definir un valor por defecto que, si se quiere, puede generarse con una función. Si no se especifica este valor es null.

Serial (y las variantes bigserial y smallserial) son especiales. En realidad son de tipo int (bigint, smallint) pero generan una secuencía y poseen un valor por defecto que genera un autoincremento del valor de la tabla, evitando valores repetidos.

5.2 Optimització de consultes, ANALYZE

Eficiencia (1)

El tema de la eficiencia es muy extenso y complicado, ya que se necesita de conocimientos de como funciona una base de datos. No obstante, vamos a ver como ver o buscar para hacer que tu base de datos sea más eficiente.

La primera herramienta que hay que conocer es la instrucción EXPLAIN, especialmente su variante EXPLAIN ANALYZE. Esta instrucción permite analizar querys, analiza el rendimiento de las diferentes partes y cuanto tarda en ejecutarse. Esto permite entender las querys para modificarlas y hacerlas más eficientes.

La segunda herramienta son los INDEX. Esta herramienta guarda los datos de alguna columna de una tabla para hacer comparaciones y búsquedas más rápido.
También existe el INDEX SPATIAL, muy recomendable aplicarlo a las capas

Eficiencia (2)

En el cas de las funciones, se pueden añadir *categories* que ayudan al planificador a hacer las sentencias más eficientes. Hay tres que son muy útiles:

Immutable/stable/volatile
Estos argumentos sirven para decidir como tratará la optimización de la función.

Immutable siempre devuelve el mismo si tiene los mismos argumentos. Normalmente funciones matemáticas.

Stable devuelve lo mismo siempre que no se modifiquen los valores de la base de datos. Normalmente selects y operaciones.

Volatile (por defecto) cada vez puede devolver un valor diferente. Por ejemplo si se utiliza la hora actual en la función.

```
create or replace suma(a int, b
int) returns int as $$
begin
   return a+b;
end;
$$
  language plpgsql
  strict
  immutable
;
```

Eficiencia (3)

Parallel safe/parallel restricted/parallel unsafe
Permite especificar si se puede y como ejecutar la función en paralelo. Hacerlos mal puede generar errores extraños o resultados incorrectos.

Si solo es una operación matemática o un select de una tabla, y solo es una tabla normal, se puede utilizar parallel safe sin preocuparse de que esté mal. Cualquier llamada a una función que no sea parallel safe puede comportar problemas.

```
create or replace suma(a int, b
int) returns int as $$
begin
   return a+b;
end;
$$
  language plpgsql
  strict
  immutable
  parallel safe
;
```

Índex

6 Administració de la Base de Dades

- 6.1 Còpies de seguretat i restauració (pg_dump i pg_restore)
- 6.2 Monitorització i manteniment
- 6.3 Gestió d'usuaris ROLE
- 6.4 Gestió de permisos GRANT

6.1 Còpies de seguretat i restauració (pg_dump i pg_restore)

pg_dump (copias de seguridad)

Herramienta propia de PostgreSQL que permite guardar datos de la base de datos en diferentes formatos, normalmente preparados para cargarlos en otra base de datos.

Es una herramienta muy útil para hacer copias de seguridad.

%pg_dump% -h %host% -d %db% -U %user% -v -F custom -f %db%_%fecha%.dump

https://www.postgresql.org/docs/16/app-pgdump.html

pg_restore (restaurar còpies de seguretat)

Herramienta propia de PostgreSQL que permite restaurar copias de base de datos en diferentes formatos, normalmente preparados para restaurarlos en una nueva base de datos.

Es una herramienta muy útil para restaurar copias de seguridad.

Restauración completa:
%pg_restore% -j 7 -h %host% -d %db% -U %user% -F custom %file%

Restauración de una tabla (jocs infantils)
%pg_restore% -j 7 -h %host% -d %db% -U %user% --clean -t jocs_infantils -F custom %file%

https://www.postgresql.org/docs/16/app-pgrestore.html

6.2 Monitorització i manteniment

Audit

Es una herramienta que genera triggers automáticos para guardar todo lo que se hace en una tabla, funciona como un log.

	schema_name text	table_name text	user_name text	action_tstamp timestamp with time zone	action text	original_data text	new_data text	query text
81	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:50:06.692466+02	I	[null]	(6567,0106000...	INSERT INTO "planejament"."pgo_plantes_edifi...
82	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:52:33.259296+02	I	[null]	(65545,010600...	INSERT INTO "planejament"."pgo_plantes_edifi...
83	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:53:08.819792+02	U	(65545,010600...	(65545,010600...	UPDATE "planejament"."pgo_plantes_edificabl...
84	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:55:21.298429+02	I	[null]	(5110,0106000...	INSERT INTO "planejament"."pgo_plantes_edifi...
85	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:56:59.313807+02	U	(6567,0106000...	(6567,0106000...	UPDATE "planejament"."pgo_plantes_edificabl...
86	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:56:59.313807+02	U	(65545,010600...	(65545,010600...	UPDATE "planejament"."pgo_plantes_edificabl...
87	planejament	pgo_produnditat_edificable	manel_llado	2023-08-07 13:27:38.692255+02	U	(352,01050000...	(352,01050000...	UPDATE "planejament"."pgo_produnditat_edifi...
88	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:56:59.328057+02	I	[null]	(4110,0106000...	INSERT INTO "planejament"."pgo_plantes_edifi...
89	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:56:59.328057+02	I	[null]	(6430,0106000...	INSERT INTO "planejament"."pgo_plantes_edifi...
90	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:57:41.154318+02	U	(5110,0106000...	(5110,0106000...	UPDATE "planejament"."pgo_plantes_edificabl...
91	planejament	pgo_plantes_edificables	manel_llado	2022-09-22 14:57:41.180687+02	I	[null]	(6110,0106000...	INSERT INTO "planejament"."pgo_plantes_edifi...

https://wiki.postgresql.org/wiki/Audit_trigger

5.2 Permisos

Permisos

PostgreSQL tiene un sistema de permisos propio (y diferente a otras bases de datos como Access o Oracle).

Los permisos pueden ser otorgados (o quitados), entre otros, a los siguientes objetos:

- Base de datos
- Esquema
- Tabla o vista
- Función

Cada objete tiene ciertos permisos y funcionan de maneras diferentes.

Nota:

Para ver todas las opciones buscar GRANT en la documentación de PostgreSQL.

Permisos

Para otorgar permisos a un usuario o grupo se utiliza **GRANT**.
Para quitar permisos se utiliza **REVOKE**.
Los dos tienen la misma estructura.

```
GRANT ALL ON                        SCHEMA cadastre TO gis_rw;
GRANT ALL ON ALL TABLES IN          SCHEMA cadastre TO gis_rw;
GRANT ALL ON ALL SEQUENCES IN       SCHEMA cadastre TO gis_rw;
GRANT USAGE ON                      SCHEMA cadastre TO gis_r;
GRANT SELECT ON ALL TABLES IN       SCHEMA cadastre TO gis_r;
```

https://www.postgresql.org/docs/16/sql-grant.html

5.3 Roles

Roles

Un rol es a lo que otorgan permiso. Puede ser un usuario, un grupo o los dos a la vez.

Normalmente se diseñan los permisos pensando en grupos y usuarios. Los usuarios se pueden conectar, y pueden tener permisos propios. En muchas ocasiones es mucho más útil hacer que haya ciertos grupos que tengan ciertos permisos y hacer que el usuario pertenezca al grupo, especialmente si el acceso es personal y hay muchos usuarios. Además este sistema permite hacer grupos de permisos que son más fáciles de manejar.

Roles

Internamente, para PostgreSQL, la única diferencia entre un usuario y un rol es que el usuario tiene la posibilidad de conectarse. Técnicamente puedes hacer que un usuario se le otorgue los permisos de otro usuario pero suele ser mala idea.

-- Crear roles
CREATE ROLE g_cons;
CREATE ROLE g_admin;

-- Usuarios de edición de GIS
*CREATE ROLE gisadmin login PASSWORD 'your_password' IN ROLE **g_admin**;*
-- Usuarios de consulta
*CREATE ROLE gis login PASSWORD 'otro_password' IN ROLE **g_cons**;*

Índex

7 Funcions avançades

- 7.1 Particionament de taules
- 7.2 Repliques i alta disponibilitat
- 7.3 Seguretat
- 7.4 Extensions i mòduls addicionals (PostGIS, etc.)

7.1 Particionament de taules

Particionament de les dades

La partición de tablas es una técnica de diseño de bases de datos que se utiliza para dividir una tabla grande en partes más pequeñas y manejables llamadas particiones.

Cada partición es esencialmente una tabla separada que almacena un subconjunto de los datos originales.

Esta técnica puede mejorar significativamente el rendimiento de las consultas y la gestión de datos para grandes conjuntos de datos.

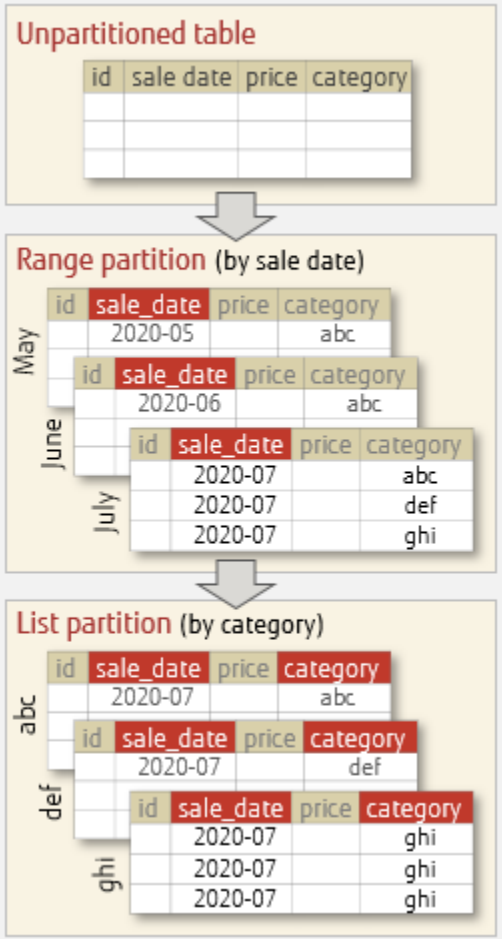
Particionament de les dades

Beneficios de la partición de tablas

1- Rendimiento de consultas mejorado:
2- Gestión de datos más sencilla:
3- Carga e indexación de datos mejoradas
4- Almacenamiento rentable

Métodes:

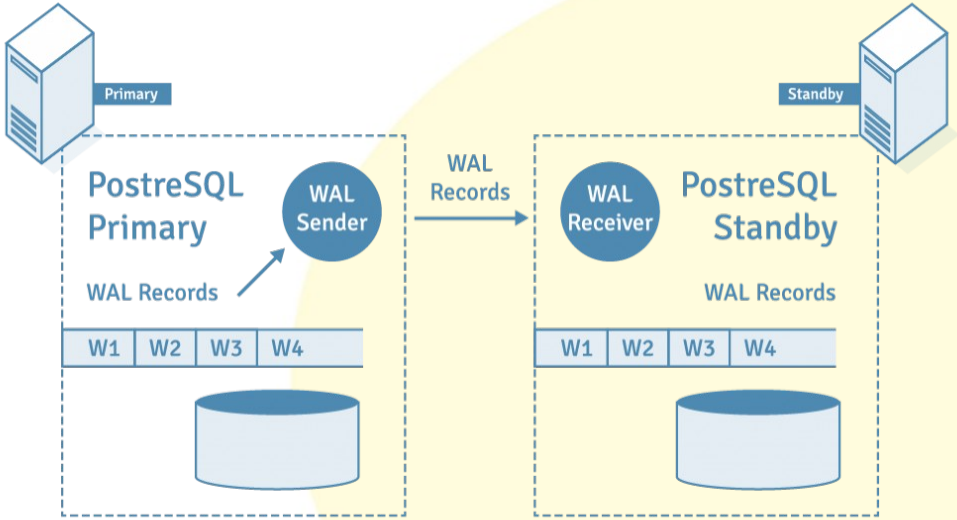
1- Range Partitioning
2- List Partitioning
3- Hash Partitioning



7.2 Repliques i alta disponibilitat

Réplica y alta disponibilidad

Los servidores de bases de datos pueden trabajar juntos para permitir que un segundo servidor asuma el control rápidamente si el servidor principal falla (alta disponibilidad) o para permitir que varias computadoras sirvan los mismos datos (balanceo de carga). Idealmente, los servidores de bases de datos podrían funcionar juntos sin problemas.



7.3 Seguretat

Seguridad

En una base de datos es muy importante la seguridad. No solo por ataques exteriores, sinó también para evitar errores humanos. Por defecto PostgreSQL es muy seguro, pero hay unas cuantas acciones que se pueden hacer para mejorar la seguridad:

- Una cuenta por persona (no compartir el usuario).
- Tener contraseñas complicadas y guardarlas en **KeePass2** o LastPass.
- Solo dar permisos a aquello que sea necesario hacer. Ni más ni menos.
- Revisar los permisos periódicamente.
- Quitar los permisos cuando alguien deje de trabajar en alguna cosa.
- Utilizar encriptación.
- Utilizar **pg_service.conf**, es mejor que tenerlo en el proyecto y permite crear uno para cada usuario.
- Para lecturas para aplicaciones utilizar un usuario de la aplicación donde se ejecuten funciones con las sentencias dentro.

7.4 Extensions i mòduls addicionals

PostGIS

- PostGIS es un módulo que añade soporte de objetos geográficos o espaciales a la base de datos relacional PostgreSQL, y la convierte en una base de datos espacial.



- PostGIS añade 3 características:
  - Tipos de datos espaciales
  - Funciones que operan con los datos espaciales
  - Indexación espacial

pgRouting



Es una librería para hacer todo tipo de cálculo de rutas.
http://pgrouting.org/

pgRouting library contains following core features:

- Dijkstra Algorithm
- Johnson's Algorithm
- Floyd-Warshall Algorithm
- A* Algorithm
- Bi-directional Algorithms * Bi-directional Dijkstra * Bi-directional A*
- Traveling Sales Person
- Driving Distance
- Turn Restricted Shortest Path (TRSP)
- many more!!!

UnAccent

Es un diccionari de text que permet eliminar els accents (signes diacrítics) dels lexemes.
Aquesta extensió s'instal·la de sèrie amb el PostgreSQL.

– Primer activem l'extensió
create extension if not exists unaccent;

– Exemple:
Select unaccent(c.nviam) FROM carrerer.vies;

Carrer Rosello --- (Carrer Roselló)

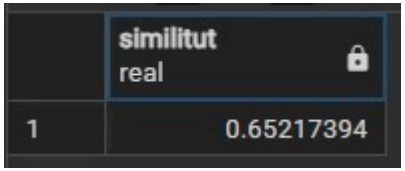
À	A
Á	A
Â	A
Ã	A
Ä	A
Å	A
Æ	AE

pg_trgm

El módulo pg_trgm proporciona funciones y operadores para determinar la similitud de texto alfanumérico basándose en la coincidencia de trigramas.
Un trigrama es un grupo de tres caracteres consecutivos tomados de una cadena.

– Primer activem l'extensió
create extension if not exists pg_trgm;

– Exemple:
Select similarity ('Av Diagonal, 137' , 'Avinguda Diagonal, 137') as similitut;



	similitut real
1	0.65217394

Gràcies per la vostra assistència

Curs de PostgreSQL



PSIG
Implementació, gestió i formació SIG

Carlos López Quintanilla
Consultor SIG

+34 699 680 261
carlos.lopez@psig.es
www.psig.es